

Digital 360

Smart HAS — AI Logistics Extension

Fase 5 — Back-end Spring Boot + Dashboard Angular

Repositório github.com/DevRafonalde/digital360_flutter

Vídeo *[link do YouTube — não listado]*



Eduardo Andrade Martins Vasques

RM 556970



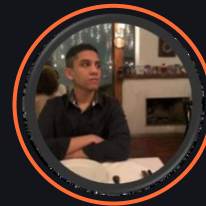
Otávio Ramos dos Santos Souza

RM 550361



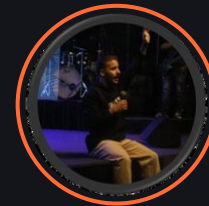
Enzo Miranda Ward de Paiva

RM 557632



Rafael Pinto de Albuquerque

RM 559136



Guilherme Leoni Vidigal Tibúrcio

RM 557500

CONTEXTO

O que já existe, antes desta fase

Digital 360 é o app de inclusão digital para idosos e público com baixo letramento — trilhas de aprendizagem, guia de serviços públicos e um assistente de dúvidas.

AI Logistics Extension é a camada construída com a Leroy Merlin no Enterprise Challenge: risco de entrega, recomendação e detecção de picos de demanda.

2 back-ends reais (Python + Java), 1 dashboard administrativo e 1 app Flutter maduro compõem o ecossistema Smart HAS hoje.

33

endpoints no backend Java

30+

telas Flutter

85

testes automatizados

Por que manter Flutter, não migrar para React Native

“O ganho técnico desta fase vem de profundidade — back-end real completo, dashboard e refactor de configuração — não de trocar o framework de UI.”

Base já madura

Acessibilidade, LGPD e 85 testes automatizados já implementados no Flutter.

Risco vs. ganho

Migrar tudo agora arriscaria regressão sem ganho técnico proporcional.

Onde o esforço rende mais

Back-end Java completo + dashboard administrativo, não a camada de UI.

Roadmap tecnológico

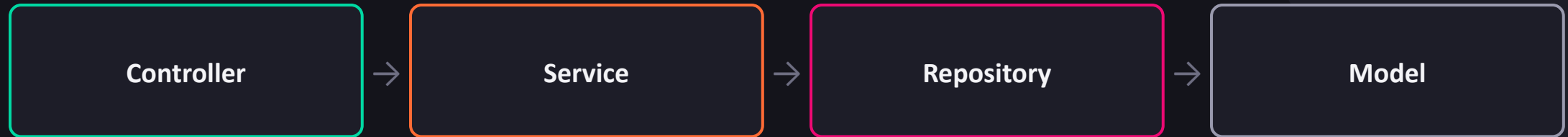


- Kotlin → Flutter (Fase 4)
- Redesign visual + LGPD
- Backend Python real
- Comunidade e gamificação

- Backend Java / Spring Boot completo
- Dashboard Angular
- Seletor de ambiente no app

- Convergência dos back-ends
- Motores de risco/recomendação com ML
- Testes JUnit / Karma

Arquitetura em camadas



Autenticação JWT stateless (access + refresh), perfis ADMIN / USUARIO, autorização por dono do recurso.

Persistência & docs H2 persistente via Spring Data JPA. Swagger / OpenAPI gerado automaticamente.

Domínio 13 controllers · 5 motores heurísticos isolados (Risco, Recomendação, Tendência, Rascunho, Gamificação).

Validado rodando de verdade — mvn spring-boot:run + chamadas reais via curl (login, CRUD, risco, comunidade, cuidador, LGPD).

Autorização por perfil e por dono

ADMIN

Escreve o catálogo — serviços e pedidos

USUARIO

Lê tudo; escreve o que é seu

Tutor

Publica curso comunitário

Dono do recurso

Vê só as próprias recomendações e gamificação

Senha protegida com BCrypt · validação de entrada · tratamento de erro centralizado no
GlobalExceptionHandler.

springdoc-openapi · /swagger-ui.html

POST	/auth/usuarios/login
GET	/cursos
POST	/entregas/{id}/recalcular-risco
POST	/pedidos/{id}/reagendar
GET	/gamificacao/ranking
GET	/tendencias

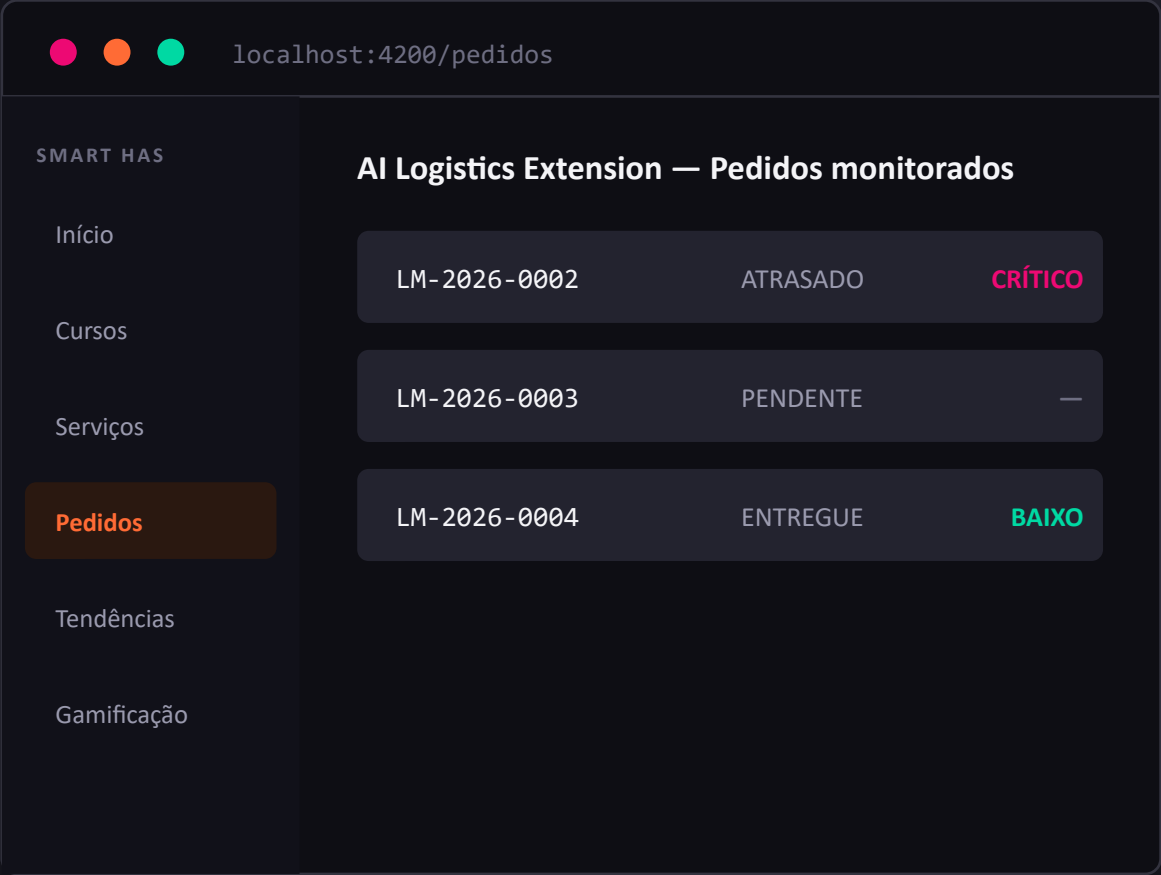
33 endpoints documentados automaticamente

Dashboard administrativo

HttpClient com interceptor de Bearer token e guard de rota protegendo o painel.

Telas: Login, Home, Cursos, Serviços, Pedidos — CRUD completo, mais Tendências e Gamificação.

Data binding: interpolação {{ }}, property [], event (), two-way [()];
diretivas *ngIf / *ngFor.



recálculo de risco e reagendamento em uma linha, direto na tela

A SEGUIR

Demonstração ao vivo

Backend (Swagger) + Dashboard + App, rodando de verdade, sobre a mesma API.



Lições e limitações

Contrato conferido contra o código real

campo senhaUser (não senha), regra isTutor — testado contra o app e o backend Python.

Implementação paralela, não substituição

O Java cobre os 33 endpoints; o Python do time continua existindo em paralelo.

Próximo passo

Hardening final com testes JUnit / Karma, planejado para a próxima rodada.

33

endpoints cobertos

CORRIGIDO

LazyInitializationException ao serializar topicosModulos — corrigida (fetch EAGER)

CORRIGIDO

JSON malformado devolvia 500 em vez de 400 — corrigido no GlobalExceptionHandler

ENCERRAMENTO

Tecnologia e inclusão, andando juntas.

Obrigado!

Repositório github.com/DevRafonalde/digital360_flutter

Vídeo *[link do YouTube — não listado]*